登录流程说明（当前项目机器人压测）

> 本文档面向质量保障中心，整理当前项目登录压测案例的协议、组包、流程和注意事项。

一、协议组包结构说明

1.1 网络层

当前登录案例默认使用 **TCP** 连接游戏网关。

1.2 收发包格式

业务消息使用 **Protobuf** 序列化，外层包由 `msgpackager` 负责组包和解包。

TCP 包格式如下：

```
| msgLen(2 bytes, little-endian) | msgId(4 bytes, little-endian) | msgBody(bytes) |
|------------- 明文包头 ----------|---------------- AES-CTR 加密包体 ----------------|
```

字段说明：

字段	长度	说明
`msgLen`	2 字节	业务 Protobuf 消息体长度，不包含 `msgId`
`msgId`	4 字节	消息 ID，小端序写入，属于加密包体的一部分
`msgBody`	变长	具体业务 Protobuf 序列化内容，属于加密包体的一部分

当前没有扩展头：`extLenSize = 0`，所以包头中没有 `extLen` 字段。

1.3 加密说明

TCP 登录案例启用 AES 加密：

```
EnabledCrypto: true
Aes_key_tcp:   "S131$#@31d%$@fklja&^jf`1-u97*&8S"
```

加密算法为 **AES-CTR**，IV 固定为：

```
73 65 42 37 24 46 35 53 23 75 66 61 6d 55 6d 41
```

加密范围是包体部分：`msgId + msgBody`；`msgLen` 包头保持明文。

1.4 组包步骤

1. 构造业务 Protobuf 消息，例如 `CReqLogin`。
2. 使用 Protobuf 将业务消息序列化为 `msgBody`。
3. 写入 2 字节小端 `msgLen = len(msgBody)`。
4. 写入 4 字节小端 `msgId`。
5. 追加 `msgBody`。
6. 对 `msgId + msgBody` 做 AES-CTR 加密。
7. 将完整二进制包写入 TCP 连接。

1.5 解包步骤

1. 从 TCP 连接读取 2 字节小端 `msgLen`。
2. 再读取 `4 + msgLen` 字节包体。
3. 对包体做 AES-CTR 解密。
4. 读取 4 字节小端 `msgId`。
5. 剩余内容作为 `msgBody`。
6. 根据 `msgId` 分发到对应响应处理函数。
7. 使用对应 Protobuf 响应结构反序列化 `msgBody`。

二、机器人初始化

2.1 账号生成

账号来源分两种：

模式	参数	说明
文件账号	`--account-on true --account-filename account.txt`	从文本文件读取账号列表
前缀账号	`--account-on false --account-prefix test --account-idx 1`	按前缀和下标生成账号，如 `test1`

当前代码中的登录请求实际使用 `Robot.Index` 作为 `userID`，并未直接发送 `PTName`。

2.2 连接初始化

每个机器人初始化时会设置：

配置	当前值/来源	说明
`TcpSvr`	`--Tcp-Svr`	TCP 网关地址，如 `127.0.0.1:8080`
`EnabledCrypto`	`true`	启用加密收发包
`Packager`	小端、2 字节长度、无扩展头	TCP 包组包器
`Aes_key_tcp`	`S131$#@31d%$@fklja&^jf`1-u97*&8S`	TCP AES 密钥

连接成功后会启动三个后台协程：

1. 收包协程：循环读取 TCP 包并调用 `OnReadPb` 分发。
2. 心跳协程：当前只保留定时框架，实际心跳发送代码被注释。
3. 异步监控协程：检查请求超时，并记录失败。

三、登录压测案例

当前项目注册了两个登录案例：

Case	中文名	说明
`LoginOnline`	登入在线	持续循环执行登录、鉴权、拉取用户信息
`LoginOnce`	登入一次在线	只执行一次登录请求，然后保持在线直到用例结束

案例注册位置：`CaseDefine.go`。

3.1 LoginOnline

流程如下：

1. 创建机器人实例。
2. 进入登录集合点，等待本 Pod 内机器人全部就绪。
3. 建立 TCP 连接。
4. 在压测持续时间内循环执行：
   - `CReqLogin`
   - `CReqAuth`
   - `CReqUserInfo`
5. 每轮循环间隔 10 毫秒。
6. 每次发送前受 `--login-ratelimiter` 限流控制。

3.2 LoginOnce

流程如下：

1. 创建机器人实例。
2. 进入登录集合点，等待本 Pod 内机器人全部就绪。
3. 建立 TCP 连接。
4. 限流允许后只发送一次 `CReqLogin`。
5. 保持循环等待，直到压测上下文结束。

四、登录相关协议

4.1 玩家登录：CReqLogin / CRespLogin

proto 文件： `gameproto/cmsg/client_msg.proto`

请求消息： `CReqLogin`

```
message CReqLogin {
    uint64 userID = 1; // 登录账号ID
    string token = 2;  // 临时登录票据
}
```

当前机器人填充值：

```
CReqLogin{
    UserID: uint64(r.Index),
    Token:  "qwertyuiop",
}
```

字段	当前值	说明
`userID`	`Robot.Index`	由账号分配器生成的账号下标
`token`	`qwertyuiop`	当前 demo 固定写死的临时票据

请求 MsgId： `3750207756`

响应 MsgId： `203388982`

响应消息： `CRespLogin`

```
message CRespLogin {
    uint32 errCode = 1;
    string errMsg = 2;
    uint64 userID = 3;
    gamedef.User user = 4;
    bool isReconnect = 5;
    string clientSessionKey = 6;
    string clientReconnectToken = 7;
    repeated gamedef.SendClientActivityData activityData = 8;
}
```

当前响应处理逻辑：

1. 使用 `CRespLogin` 反序列化响应包体。
2. 反序列化失败则标记 `CReqLogin` 失败，错误码记为 `1001`。
3. 反序列化成功则标记 `CReqLogin` 成功。

当前代码未检查 `errCode` 是否为 0，仅以 Protobuf 反序列化是否成功作为请求成功依据。

4.2 账号鉴权：CReqAuth / CRespAuth

proto 文件： `gameproto/cmsg/client_msg.proto`

请求消息： `CReqAuth`

```
message CReqAuth {
    string account = 1;              // 账号
    string password = 2;             // 密码
    gamedef.ExtraAccountInfo extra = 3;
    bool isReconnect = 4;
    uint64 reconnectUserID = 5;
    string reconnectToken = 6;
    string ticket = 7;
    string sign = 8;
    string token = 9;
    int64 timestamp = 10;
}
```

当前机器人发送的是空结构体：

```
CReqAuth{}
```

请求 MsgId： `1605799248`

响应 MsgId： `566867705`

响应消息： `CRespAuth`

```
message CRespAuth {
    uint32 errCode = 1;
    string errMsg = 2;
    string account = 3;
    uint64 userID = 4;
    string token = 5;
    int64 unlockTime = 6;
    int64 lockTime = 7;
}
```

当前响应处理逻辑：

1. 使用 `CRespAuth` 反序列化响应包体。
2. 反序列化失败则标记 `CReqAuth` 失败，错误码记为 `1002`。
3. 反序列化成功则标记 `CReqAuth` 成功。

4.3 查询用户信息：CReqUserInfo / CRespUserInfo

proto 文件： `gameproto/cmsg/client_msg.proto`

请求消息： `CReqUserInfo`

```
message CReqUserInfo {
    uint64 userID = 1;        // 玩家角色ID
    uint32 fromGateID = 2;
    uint64 fromSessionID = 3;
    gameconf.GetUserInfoTyp typ = 4;
}
```

当前机器人发送的是空结构体：

```
CReqUserInfo{}
```

请求 MsgId： `876549255`

响应 MsgId： `874691946`

响应消息： `CRespUserInfo`

```
message CRespUserInfo {
    uint32 errCode = 1;
    string errMsg = 2;
    uint64 userID = 3;
    gamedef.UserBrief userBrief = 4;
    gamedef.GameRecordClient gameRecord = 5;
    repeated gamedef.GameResultRecord gameResultRecord = 6;
    repeated gamedef.FinishAchievement finishAchievement = 7;
    repeated gamedef.AchievementProgress achievementProgress = 8;
    repeated gamedef.AthleticsRecord athleticsRecord = 9;
    repeated gamedef.Task masterTask = 10;
    gamedef.FightLandlordRecord fightLandlordRecord = 11;
    repeated gamedef.CommonModeGameResultRecordData commonModeGameResultRecordData = 12;
    uint32 insigniaScore = 13;
}
```

当前响应处理逻辑：

1. 使用 `CRespUserInfo` 反序列化响应包体。
2. 反序列化失败则标记 `CReqUserInfo` 失败，错误码记为 `1003`。
3. 反序列化成功则标记 `CReqUserInfo` 成功。

五、消息 ID 速查表

消息名	请求 MsgId	响应 MsgId	方向	当前处理函数
`CReqLogin / CRespLogin`	`3750207756`	`203388982`	CS/SC	`CReqLogin` / `CRespLogin`
`CReqAuth / CRespAuth`	`1605799248`	`566867705`	CS/SC	`CReqAuth` / `CRespAuth`
`CReqUserInfo / CRespUserInfo`	`876549255`	`874691946`	CS/SC	`CReqUserInfo` / `CRespUserInfo`

六、完整登录流程图

6.1 LoginOnline

```
机器人
  |
  ├─ 根据账号配置生成 Robot.Index / PTName
  |
  ├─ 等待 LoginRendezvous 集合点
  |
  ├─ TCP 连接 --Tcp-Svr ──────────────────────────> Game Gateway
  |
  ├─ 启动收包协程 / 心跳协程 / 异步监控协程
  |
  ├─ 循环直到压测 Duration 结束
  |   |
  |   ├─ 限流器 LoginRate.Allow()
  |   |
  |   ├─ 发送 CReqLogin{userID: Robot.Index, token:"qwertyuiop"}
  |   |    └─ 收到 CRespLogin ──────────────────> 标记 CReqLogin 成功/失败
  |   |
  |   ├─ 发送 CReqAuth{}
  |   |    └─ 收到 CRespAuth ──────────────────> 标记 CReqAuth 成功/失败
  |   |
  |   └─ 发送 CReqUserInfo{}
  |        └─ 收到 CRespUserInfo ──────────────> 标记 CReqUserInfo 成功/失败
  |
  └─ 上下文结束，机器人退出用例循环
```

6.2 LoginOnce

```
机器人
  |
  ├─ 根据账号配置生成 Robot.Index / PTName
  |
  ├─ 等待 LoginRendezvous 集合点
  |
  ├─ TCP 连接 --Tcp-Svr ──────────────────────────> Game Gateway
  |
  ├─ 限流器 LoginRate.Allow()
  |
  ├─ 只发送一次 CReqLogin{userID: Robot.Index, token:"qwertyuiop"}
  |    └─ 收到 CRespLogin ──────────────────────> 标记 CReqLogin 成功/失败
  |
  └─ 保持在线，直到压测 Duration 结束
```

七、启动参数示例

单机器人本地调试示例：

```
go run . \
  --total-robot-num 1 \
  --pod-num 1 \
  --pod-id 0 \
  --log-on true \
  --log-level 2 \
  --account-on false \
  --account-filename account.txt \
  --account-prefix test \
  --account-idx 1 \
  --case LoginOnline \
  --case-robot-num 1 \
  --pod-robot-num 1 \
  --timeout 30 \
  --ramp-up-period 0 \
  --duration 30 \
  --count 1 \
  --Tcp-Svr 127.0.0.1:8080 \
  --creator "侯宇祺" \
  --login-ratelimiter 1
```

常用参数说明：

参数	说明
`--case`	压测案例名，当前支持 `LoginOnline`、`LoginOnce`
`--pod-robot-num`	当前 Pod 内机器人数量
`--duration`	压测持续时间，单位秒
`--timeout`	单次异步请求等待超时时间，单位秒
`--Tcp-Svr`	TCP 网关地址
`--login-ratelimiter`	登录限流阈值，按 Pod 数拆分到各 Pod
`--account-prefix`	自动生成账号名前缀
`--account-idx`	当前批次账号起始下标

八、注意事项

1. **当前用例依赖 TCP 地址**：`RobotInit` 当前调用 `CreateConn()`，必须传入可连接的 `--Tcp-Svr`。
2. **必须初始化登录限流器**：`LoginOnline` 和 `LoginOnce` 中直接调用 `global.LoginRate.Allow()`，建议始终传入 `--login-ratelimiter` 且值大于 0。
3. **当前成功判断较宽松**：响应处理函数只判断 Protobuf 反序列化是否成功，暂未判断业务 `errCode`。
4. **心跳未实际发送**：心跳协程存在，但 `HeartBeatReq()` 调用被注释，长连接保活依赖服务端和当前连接策略。
5. **Token 是固定 demo 值**：`CReqLogin.Token` 当前固定为 `qwertyuiop`，如果目标环境要求真实票据，需要补充鉴权/取票逻辑。
6. **账号并发需唯一**：压测时应保证每个机器人使用不同 `Robot.Index`，避免同账号互踢或状态覆盖。
7. **私有依赖构建要求**：项目依赖 `gitlab.qa.dobest.com/qa/yokamancli`，本地构建可能需要公司网络和 `GOPRIVATE` 配置。